

Άσκηση 4

Θέματα Όρασης Υπολογιστών

Περιεχόμενα

Μέρος Α	2
1).....	2
2).....	3
3).....	5
4).....	10
5).....	13
6).....	17
Μέρος Β	18
2).....	18
3).....	19
a).....	19
b).....	20
c).....	25
d).....	27
4).....	30
5).....	31

Ο κώδικας μπορεί να βρεθεί εδώ:

<https://github.com/GrigorisTzortzakis/Computer-Vision/tree/main/Exercise%204>

Μέρος Α

1)

Το script `ecc_lk_alignment` έχει ως στόχο να φορτώσει δύο εικόνες, την `template` (πρότυπη εικόνα) και την `image` (εικόνα που θέλουμε να ευθυγραμμίσει), και στη συνέχεια να εφαρμόσει τις δύο μεθόδους ευθυγράμμισης, τον `ecc` και το `lk`.

Τα ορίσματα εισόδου του κώδικα είναι:

`image`: Η εικόνα που θέλουμε να μετασχηματίσουμε.

`template`: Ο στόχος ευθυγράμμισης μας.

`levels`: Ο αριθμός των επιπέδων στην πυραμίδα εικόνων.

`noi`: Ο αριθμός επαναλήψεων που θα εκτελεστούν σε κάθε επίπεδο της πυραμίδας.

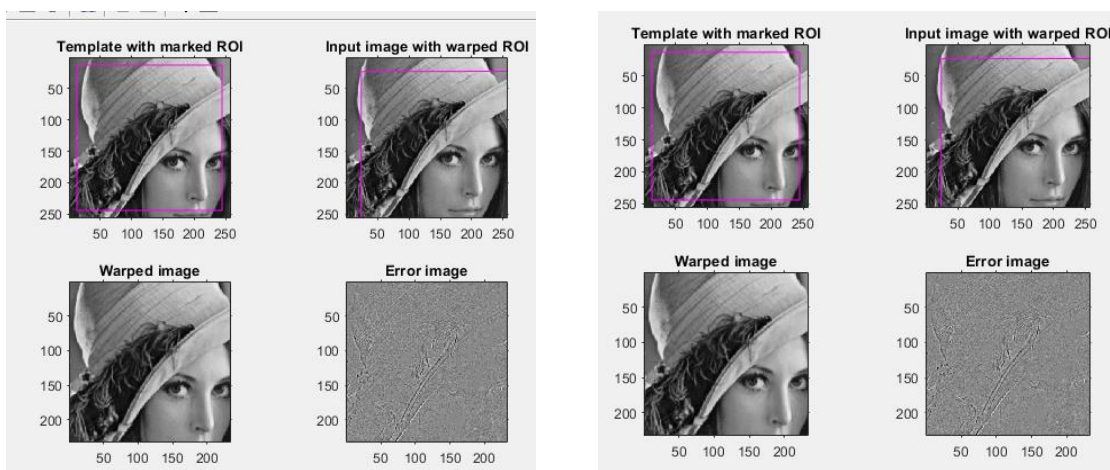
`transform`: Το είδος του μετασχηματισμού (affine ή homography).

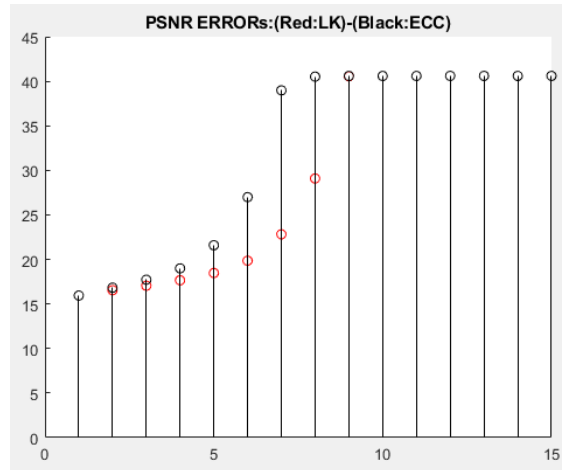
`delta_rho_init`: Προαιρετική αρχικοποίηση του μετασχηματισμού (μητρώο 2×3 για affine ή 3×3 για homography).

Όσον αφορά την έξοδο, λαμβάνουμε γραφήματα σχετικά με την απόδοση των αλγορίθμων και τα αποτελέσματα του μετασχηματισμού στην εικόνα.

Κάνουμε την παραδοχή ότι το `delta_rho_init` δεν θα είναι όλο μηδενικά καθώς για κάποιον λόγο δεν δέχεται τότε το `rho` σωστά τα ορίσματα και δεν γίνεται ταυτοτικό μητρώο, άρα δεν πραγματοποιείτε ο μετασχηματισμός. Επίσης, τρέχουμε τον αλγόριθμο με `level=1` καθώς δεν μας παρέχεται η συνάρτηση `next level` που χρειάζεται για παραπάνω `level`.

Για το παράδειγμα μας συγκρίνουμε το πρώτο frame με το frame 20 του βίντεο 1 `high`. Επίσης, θέτουμε 15 επαναλήψεις και το `delta_rho_init` ως ταυτοτικό μητρώο.





Βλέπουμε ότι και οι 2 αλγόριθμοι πετυχαίνουν τον μετασχηματισμό της εικόνας στο σωστό Roi. Το σφάλμα πρακτικά είναι ίδιο, αφού και οι 2 αλγόριθμοι συγκλίνουν.

Παρατηρούμε πως ο ecc συγκλίνει πιο γρηγορά και έχει υψηλότερο psnr στις ίδιες επαναλήψεις.

2)

`spatial_interp.m`:

Η συνάρτηση `spatial_interp.m` υλοποιεί τη διαδικασία του *inverse warping* (αντίστροφης παραμόρφωσης). Στην πράξη, όταν ευθυγραμμίζουμε μια εικόνα ως προς μία άλλη, χρειάζεται να γνωρίζουμε τη φωτεινότητα κάθε σημείου της εικόνας αφού αυτή παραμορφωθεί (μέσω *affine* ή *homography*). Επειδή οι συντεταγμένες μετά τον μετασχηματισμό συνήθως δεν αντιστοιχούν σε ολόκληρους (*integer*) δείκτες αλλά σε *subpixel*, απαιτείται παρεμβολή (*interpolation*) για την εύρεση της τιμής φωτεινότητας στα νέα σημεία.

Σε αυτή τη συνάρτηση, δίνονται οι τιμές των n_x , n_y , που καθορίζουν την περιοχή ενδιαφέροντος (ROI) στην οριζόντια και κατακόρυφη διεύθυνση αντίστοιχα. Δημιουργείται ένα πλέγμα συντεταγμένων (x,y) μέσω της `meshgrid(nx, ny)`, το οποίο αντιστοιχεί στα παλιά *pixel* (πριν την παραμόρφωση). Στη συνέχεια, εφαρμόζεται η μετασχηματιστική μήτρα A στις ομογενείς συντεταγμένες $(x,y,1)$ προκειμένου να βρεθούν οι νέες συντεταγμένες (x',y') μετά τον μετασχηματισμό. Αν ο μετασχηματισμός είναι *homography*, προστίθεται ένα επιπλέον βήμα κανονικοποίησης.

Η `interp2` είναι η συνάρτηση της *matlab* που αναλαμβάνει τη δειγματοληψία του πίνακα εικόνας σε αυτά τα νέα σημεία. Παρέχονται διάφοροι τρόποι παρεμβολής, αν και στο παράδειγμα δίνεται χαρακτηριστικά η *bilinear*, η οποία υπολογίζει τη φωτεινότητα ενός σημείου με βάση τη γραμμική παρεμβολή των κοντινότερων ακέραιων θέσεων.

Αφού η `interp2` υπολογίσει τις τιμές φωτεινότητας στα νέα σημεία, το αποτέλεσμα αναμορφώνεται (`reshape`) στην ίδια δισδιάστατη δομή με το μέγεθος του ROI (`length(ny)*length(nx)`). Τέλος, οποιαδήποτε μη διαθέσιμη τιμή (NaN) μηδενίζεται, καθώς οι μερικές εκτός ορίων συντεταγμένες και οι μη έγκυρες περιοχές δεν πρέπει να αφήνουν κενά. Έτσι, το `out` αποτελεί την τελική ευθυγραμμισμένη εικόνα του ROI, η οποία συγκρίνεται στην επόμενη φάση με το `template` ή χρησιμοποιείται για τον υπολογισμό του σφάλματος και την ενημέρωση των παραμέτρων του μετασχηματισμού.

`image_jacobian.m`:

Η συνάρτηση `image_jacobian.m` υπολογίζει τον λεγόμενο Ιακωβiano (G) της παραμορφωμένης εικόνας ως προς τις παραμέτρους του μετασχηματισμού. Στο πλαίσιο της ευθυγράμμισης, χρειαζόμαστε να γνωρίζουμε πώς μεταβάλλεται η ένταση της εικόνας όταν αλλάζουν οι παράμετροι, δηλαδή πώς επηρεάζονται οι τελικές τιμές φωτεινότητας σε κάθε `pixel` μετά την παραμόρφωση.

Πιο συγκεκριμένα, σε κάθε βήμα του αλγορίθμου, η παράγωγος της εικόνας υπολογίζεται στις συντεταγμένες της `warped` εικόνας, `gx` αντιστοιχεί στην οριζόντια παράγωγο και `gy` αντιστοιχεί στην κατακόρυφη παράγωγο

Παράλληλα, ο Ιακωβιανός αφορά τις μερικές παράγωγες των συντεταγμένων x, y ως προς τις παραμέτρους που περιγράφουν τον μετασχηματισμό (π.χ. περιστροφή, κλίμακα, μετατόπιση). Δεδομένου ότι χρειαζόμαστε τον Ιακωβιανό της ίδιας της εικόνας ως προς αυτές τις παραμέτρους, συνδυάζονται οι διαφορικές σχέσεις της εικόνας (gx, gy) με τις μερικές παράγωγες των συντεταγμένων (`jac`).

`warp_jacobian`:

Η συνάρτηση `warp_jacobian.m` υπολογίζει τις μερικές παράγωγους των τελικών συντεταγμένων (x', y') ως προς τις παραμέτρους του εκάστοτε μετασχηματισμού (`affine`, `homography`, `translation`, ή `euclidean`). Σε κάθε επανάληψη του αλγορίθμου ευθυγράμμισης, απαιτείται να γνωρίζουμε πώς αλλάζουν οι νέες συντεταγμένες ενός σημείου (x, y) στην εικόνα όταν μεταβάλλονται ελάχιστα οι παράμετροι του μετασχηματισμού. Αυτή η πληροφορία είναι κρίσιμη, καθώς χρησιμοποιείται στη συνέχεια από τη συνάρτηση `image_jacobian.m` για να κατασκευαστεί ο πίνακας G , ο οποίος καθορίζει πώς η ένταση της εικόνας (και οι παράγωγοί της) εξαρτώνται από τις παραμέτρους.

Συγκεκριμένα, η συνάρτηση δέχεται ως είσοδο τους διανύσματα `nx` και `ny` που καθορίζουν τις συντεταγμένες των `pixels` στην ROI, καθώς και το μητρώο `warp` που περιγράφει την τρέχουσα κατάσταση του μετασχηματισμού. Η διαφορά μεταξύ των διαφορετικών μετασχηματισμών συνίσταται στον τρόπο με τον οποίο ορίζονται οι σχέσεις μεταξύ (x, y) πριν και μετά την παραμόρφωση. Έτσι, για τον `affine` μετασχηματισμό, οι νέες συντεταγμένες υπολογίζονται γραμμικά

από τις παλιές (x',y') βάσει 6 παραμέτρων, ενώ για τον homography λαμβάνονται 8 παράμετροι (με πιο σύνθετη, προβολική εξάρτηση και ανάγκη κανονικοποίησης). Αντίστοιχα, στα translation/euclidean, ο πίνακας είναι πιο απλός γιατί έχουμε λιγότερες παραμέτρους (2 ή 3).

Με το πέρασμα από τις παλιές συντεταγμένες (x,y) στις νέες (x',y') , η `warp_jacobian` δημιουργεί τον πίνακα J , στον οποίο κάθε στήλη αντιστοιχεί σε μια παράμετρο του μετασχηματισμού, ενώ οι γραμμές αντιστοιχούν στη μερική παράγωγο για κάθε πιθανό σημείο.

`param_update`:

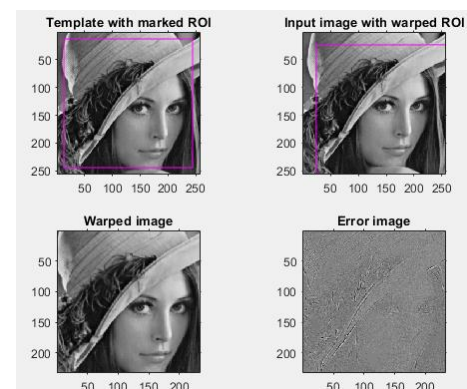
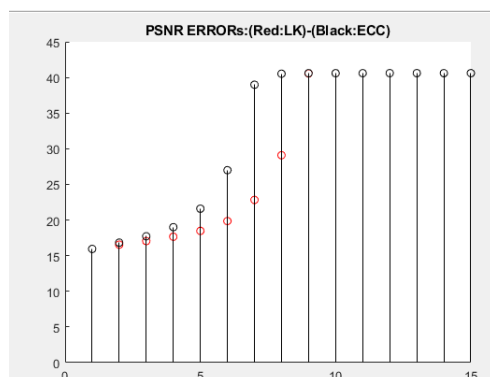
Η συνάρτηση `param_update.m` έχει ως κύριο στόχο να επικαιροποιεί τις παραμέτρους του εκάστοτε μετασχηματισμού (translation, euclidean, affine, homography) προσθέτοντας τη διόρθωση που υπολογίστηκε σε κάθε βήμα του αλγορίθμου ευθυγράμμισης. Πρόκειται για ένα κρίσιμο βήμα, καθώς καθορίζει τον τρόπο με τον οποίο ο πίνακας παραμόρφωσης προσαρμόζεται μετά την επίλυση του συστήματος εξισώσεων που στοχεύει στη μείωση του σφάλματος μεταξύ της αρχικής και της εικόνας που θέλουμε να μετασχηματίσουμε.

3)

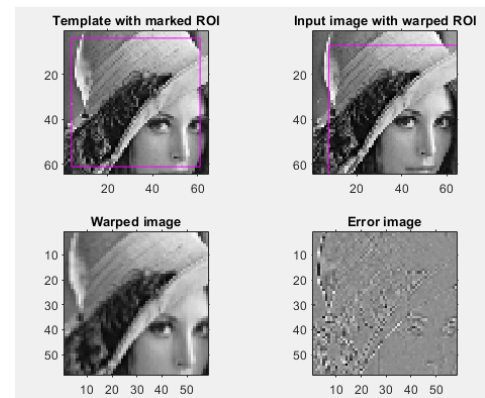
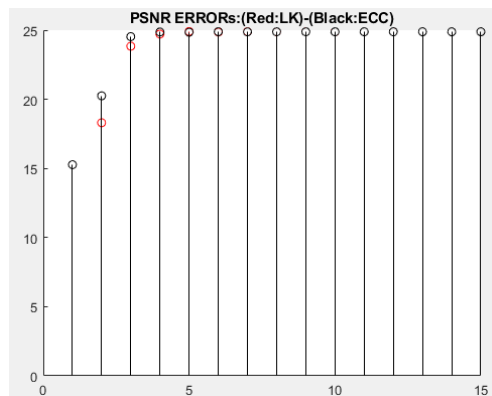
Αρχικά, συγκρίνουμε το πρώτο frame με το 20. Θέτουμε 15 επαναλήψεις και παρατηρούμε τα εξής:

Για το πρώτο βίντεο, το οποίο έχει υποστεί παραμόρφωση μετατόπισης μόνο (πηγαίνει το απλά προς τα κάτω διαγώνια δεξιά) παρατηρούμε ότι οι αλγόριθμοι συγκλίνουν πιο ομαλά στο βίντεο υψηλής ανάλυσης και πετυχαίνουν μεγαλύτερο $psnr$ (42db αντί για 38). Επίσης, ο `ecc` συγκλίνει πιο γρηγορά και έχει καλύτερο $psnr$ στις ίδιες επαναλήψεις. Το σφάλμα επίσης είναι εμφανώς μικρότερο στο βίντεο υψηλής ανάλυσης. Να σημειωθεί ότι για τις εικόνες σφάλματος είναι οι ίδιες και για τους 2 αλγορίθμους, εφόσον οι αλγόριθμοι συγκλίνουν στο ίδιο τελικό επίπεδο. Στην χαμηλή ανάλυση προφανώς συγκλίνουν νωρίτερα, αφού έχουν λιγότερη δουλειά να κάνουν, είναι περιορισμένοι από την ποιότητα.

Vid 1 high:

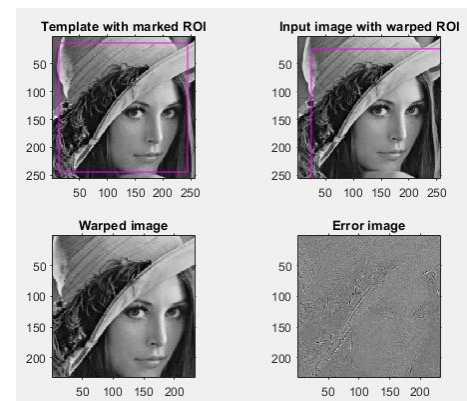
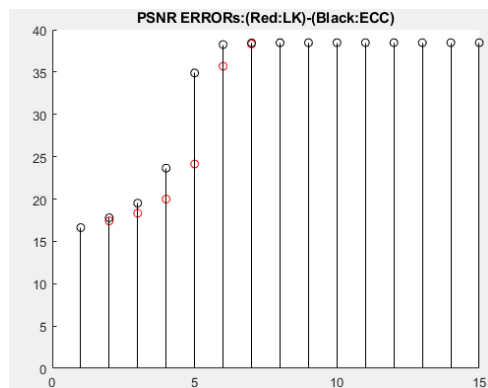


Vid 1 low:

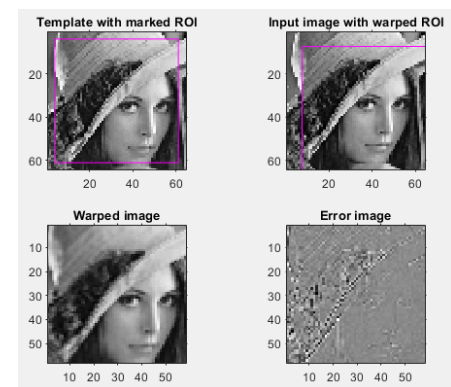
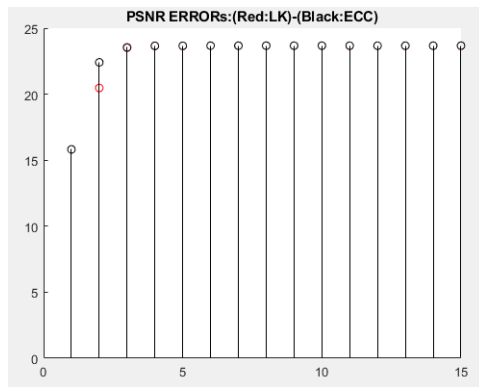


Στο βίντεο 2, το οποίο έχει υποστεί πολλαπλές παραμορφώσεις παρατηρούμε τα ίδια αποτελέσματα και επιπλέον βλέπουμε ότι τώρα οι αλγόριθμοι πετυχαίνουν μικρότερο ρ_{snr} λόγω των παραπάνω παραμορφώσεων. Τα βίντεο χαμηλής ανάλυσης συγκλίνουν πιο γρήγορά αλλά έχουν μικρότερο ρ_{snr} .

Vid 2 high:



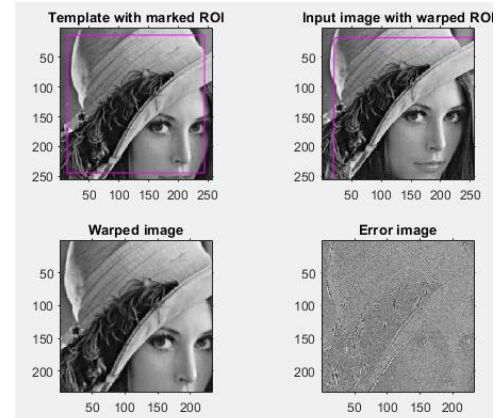
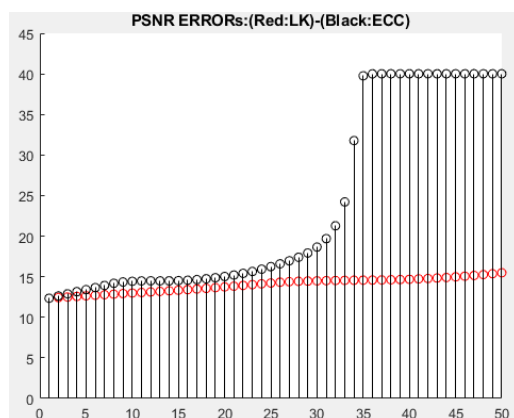
Vid 2 low:



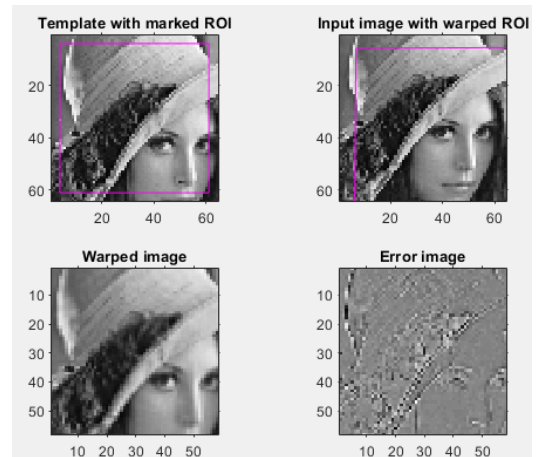
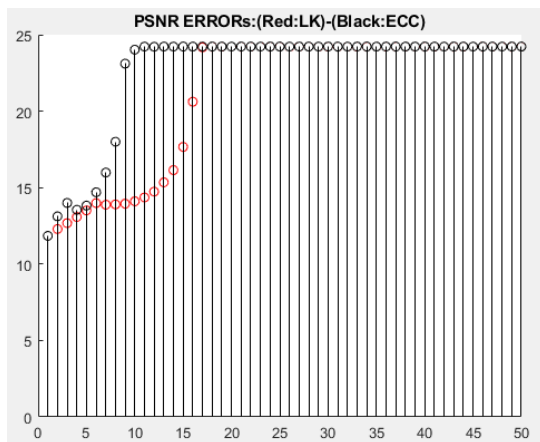
Συνεχίζοντας, συγκρίνουμε το πρώτο frame με το 60. Αυξάνουμε τις επαναλήψεις σε 50:

Τα συμπεράσματα μας είναι πιο εμφανής, όπως βλέπουμε για το βίντεο 1. Στο βίντεο high, ο ecc συγκλίνει πιο αργά σε σχέση με το Low, όμως επιτυγχάνει καλύτερο rsnr. Ο lk στο high έχει ελάχιστη βελτίωση με την πάροδο του χρόνου και απέχει αρκετά από τον ecc (δεν συγκλίνει σε 50 επαναλήψεις), ενώ στο low βελτιώνεται σημαντικά και φτάνει την απόδοση του ecc. Μια αρκετά ενδιαφέρον παρατήρηση είναι ότι ο lk θα συγκλίνει μετρά από περίπου 68 επαναλήψεις, ενώ ο ecc μετά από 34.

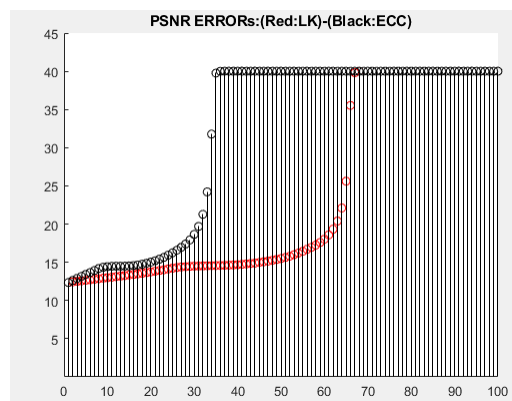
Vid 1 high:



Vid 1 low:

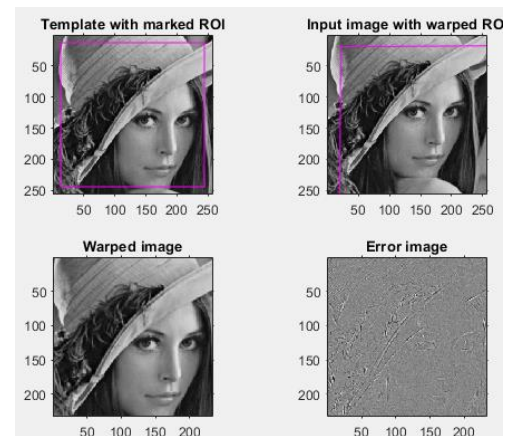
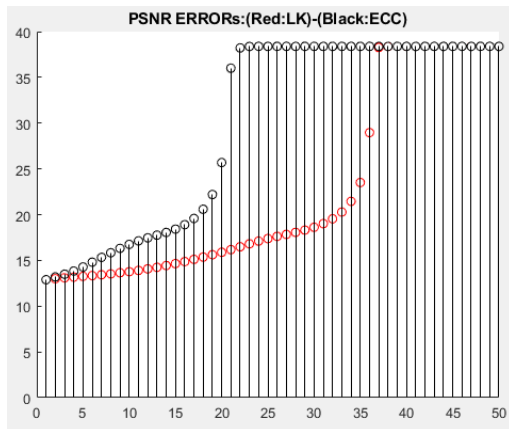


Εδώ μπορούμε να δούμε τι συμβαίνει με 100 επαναλήψεις.

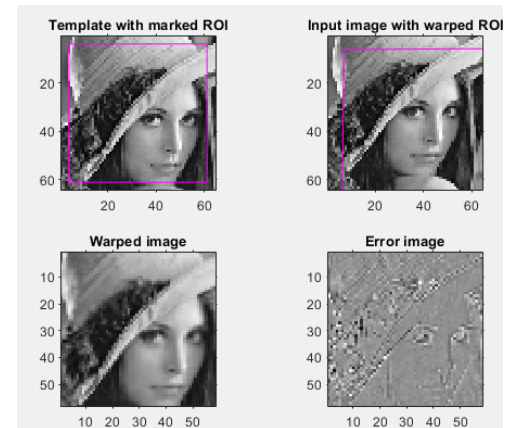
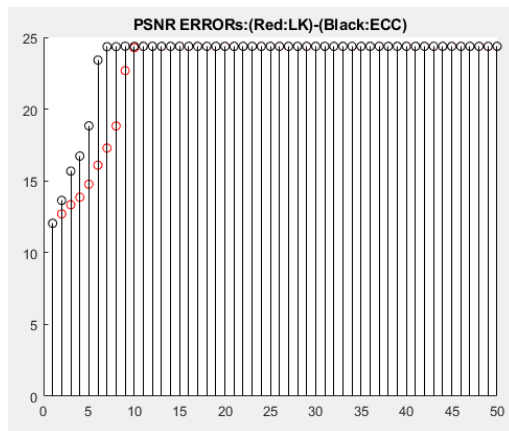


Τα ευρήματα μας είναι ίδια και για το βίντεο 2, όπου συγκλίνει νωρίτερα από το 1 αλλά έχει μικρότερο Psnr. Ο lk χρειάζεται περισσότερες επαναλήψεις για να συγκλίνει.

Vid 2 high:



Vid 2 low:



Τελικά συμπεραίνουμε ότι η ανάλυση παίζει ρόλο στην ταχύτητα σύγκλισης και στο ρ_{snr} . Όσο υψηλότερη η ανάλυση, τόσο περισσότερο αργεί να συγκλίνει αλλά έχει μεγαλύτερο ρ_{snr} . Επίσης, βλέπουμε ότι όσο περισσότερες μορφές παραμορφώσεων υπάρχουν, τόσο μικρότερο ρ_{snr} επιτυγχάνουμε αλλά οι αλγόριθμοι συγκλίνουν γρηγορότερα.

Τα ευρήματά μας σχετικά με το ότι ο ecc έχει μικρότερο σφάλμα πράγματι επαληθεύονται εξετάζοντας τα αποτελέσματα του αλγορίθμου.

```
MSE (ECC) across iterations:
35.7335 31.3693 25.5971 15.1602 3.6511 2.4862 2.4385 2.4392 2.4388 2.4389 2.4388 2.4388 2.4388 2.4388 2.4388

MSE (LK) across iterations:
35.7335 32.4589 29.2025 23.5275 13.2820 2.9353 2.4599 2.4383 2.4391 2.4388 2.4389 2.4388 2.4388 2.4388 2.4388
```

4)

Χρησιμοποιώντας το frame 1 και 2, διαπιστώνουμε ότι και οι 2 αλγόριθμοι έχουν την ίδια απόδοση όσον αφορά το σφάλμα και την ταχύτητα σύγκλισης. Αυτό συμβαίνει καθώς δεν υπάρχουν μεγάλες αλλαγές στην εικόνα μας, άρα ο lk σε αυτή την περίπτωση αποδίδει το ίδιο με τον ecc. Διαφορά αρχίζουμε να βλέπουμε κατά μέσο ορό όταν συγκρίνουμε το πρώτο frame με το 5 και μετά.

```
Final ECC warp (2x3):
0.9989 -0.0000 -0.0131
-0.0000 0.9985 -0.0310

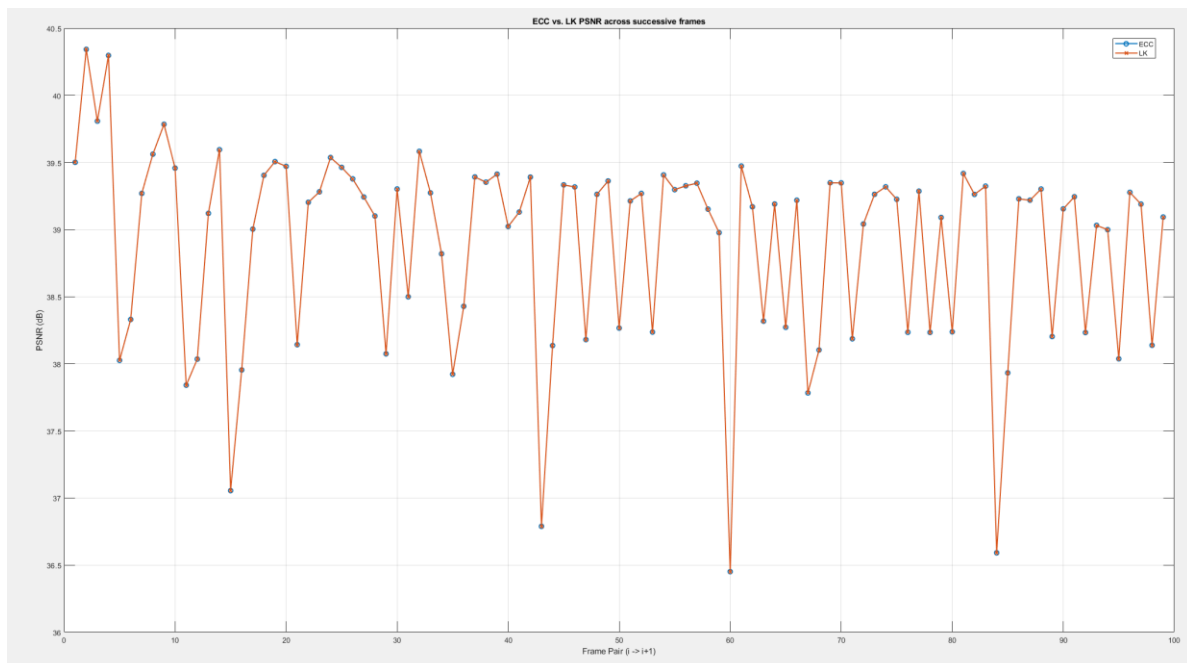
Final LK warp (2x3):
0.9989 0.0000 -0.0137
-0.0000 0.9985 -0.0319

MSE (ECC) across iterations:
5.7077 5.5152 5.4700 5.4785 5.4766 5.4771 5.4770 5.4770 5.4770 5.4770 5.4770 5.4770 5.4770 5.4770 5.4770

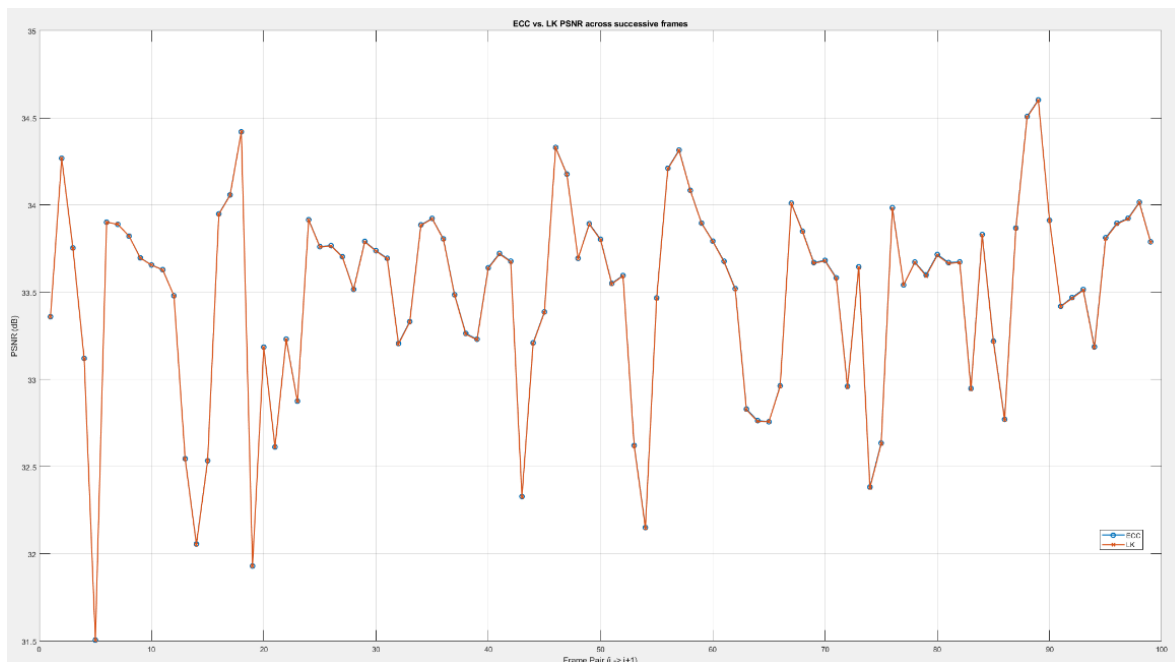
MSE (LK) across iterations:
5.7077 5.5075 5.4720 5.4789 5.4774 5.4778 5.4777 5.4777 5.4777 5.4777 5.4777 5.4777 5.4777 5.4777 5.4777

ECC correlation (rho) across iterations:
0.9958 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962 0.9962
```

Vid 1 high:



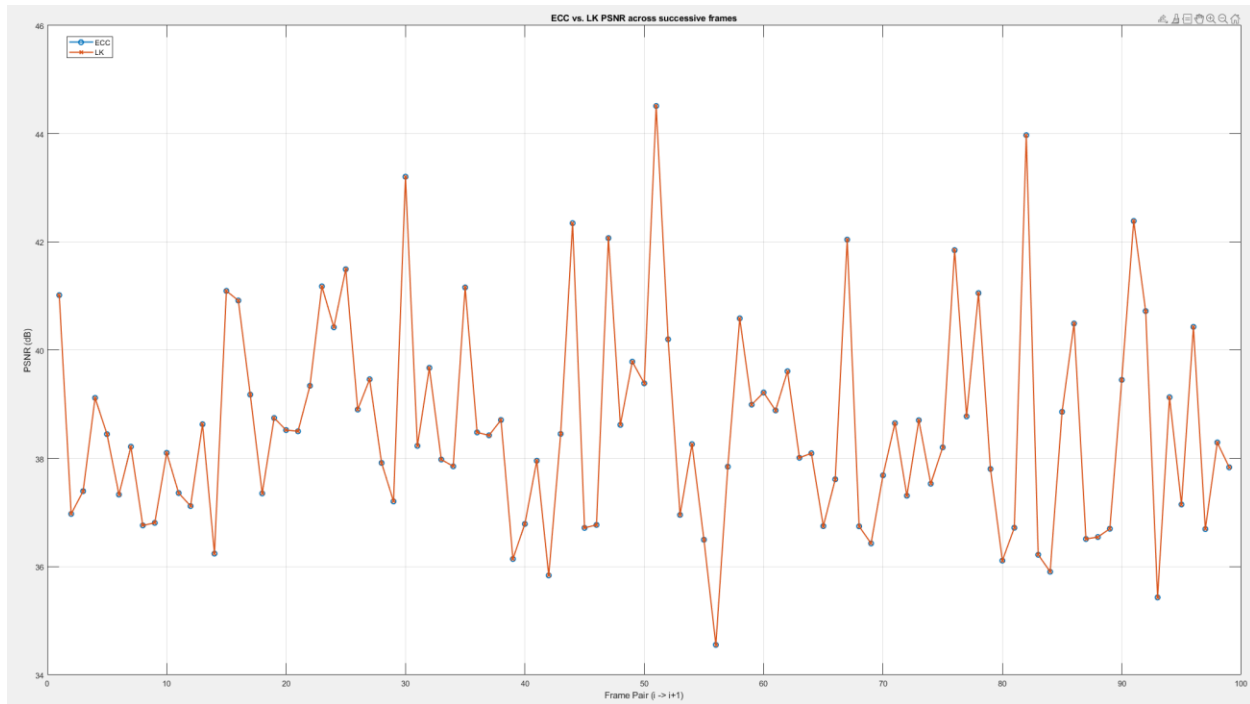
Vid 1 low:



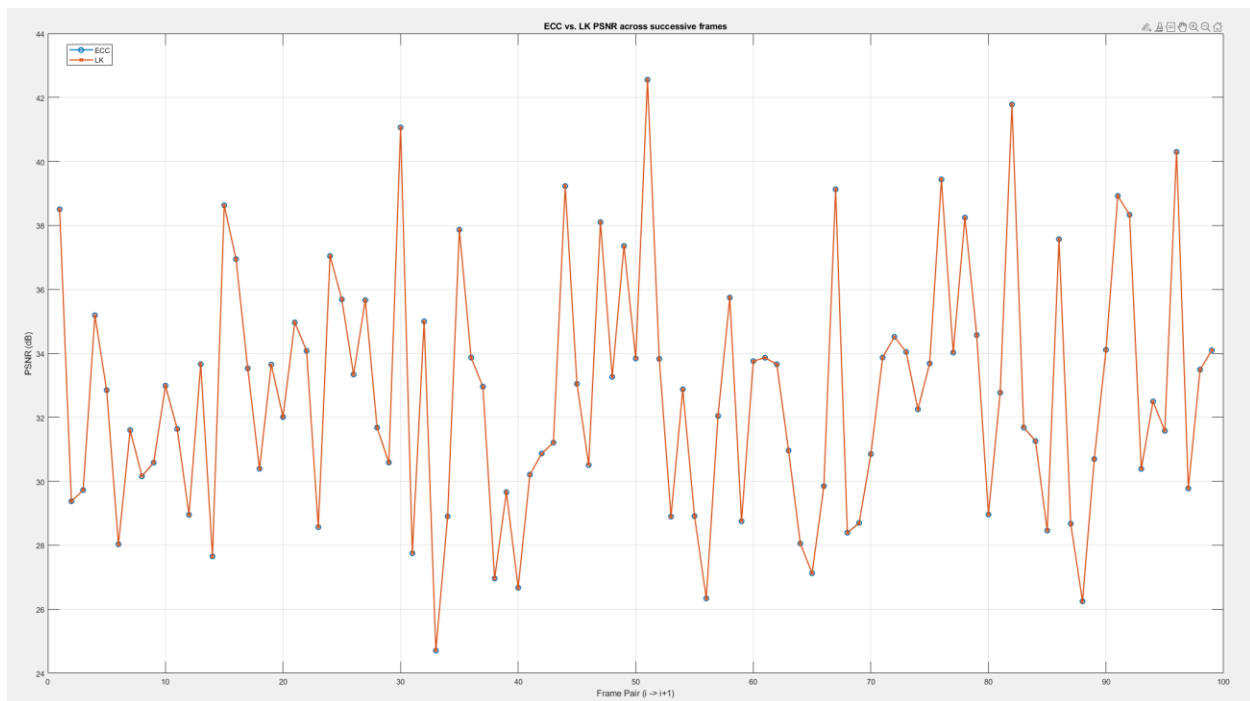
Παρατηρούμε ότι όπως έχουμε επιβεβαιώσει από το προηγούμενο ερώτημα, η υψηλότερη ανάλυση πετυχαίνει μεγαλύτερο Psnr. Στην χαμηλότερη ανάλυση, οι αλλαγές μεταξύ των frame είναι πιο ομαλές, αφού είναι πιο εύκολη η ανίχνευση τους, όμως έχουν μικρότερη ακρίβεια. Οι αυξομειώσεις δημιουργούνται καθώς μερικά frame έχουν μικρότερη παραμόρφωση σε σχέση με άλλα, άρα έχουμε καλύτερα αποτελέσματα.

Οι ίδιες παρατηρήσεις ισχύουν και για το βίντεο 2, με την διαφορά ότι οι αυξομειώσεις είναι ακόμα πιο απότομες λόγω πιο συνθέτων παραμορφώσεων που περιέχει. Το βίντεο 1 έχει μόνο μετατόπιση διαγώνια κάτω προς τα δεξιά, ενώ το βίντεο 2 περιέχει επιπλέον και κούνημα της κάμερας.

Vid 2 high:



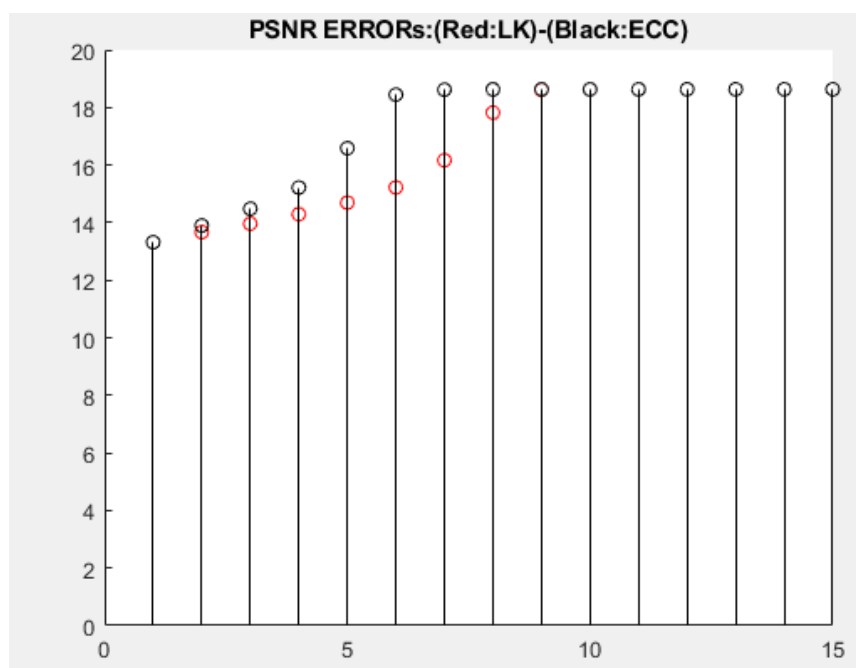
Vid 2 low:



5)

Template:

Στην πρώτη περίπτωση αυξήσαμε το contrast και το brightness, ενώ στην δεύτερη τα μειώσαμε στο template. Παρατηρούμε ότι στην πρώτη περίπτωση οι αλγόριθμοι έχουν σχεδόν την ίδια επίδοση, ενώ στην δεύτερη απέχουν αρκετά μετά από μερικές επαναλήψεις. Και τα δυο είναι για το vid 1 high. Τα συμπεράσματα συμπίπτουν με την θεωρία, καθώς ο ecc είναι σχεδιασμένος να είναι ανεξάρτητος σε φωτομετρικές αλλαγές, ενώ ο lk εξαρτάται από την φωτεινότητα. Γενικά, αλλάζοντας τιμές της φωτεινότητας βλέπουμε ότι ο ecc έχει σταθερό διάγραμμα ενώ ο lk μεταβάλλεται σημαντικά. Ο ecc συγκλίνει με ίδια ταχύτητα ότι αλλαγές και να κάνουμε, ενώ ο lk μεταβάλλεται σημαντικά. Τα ίδια συμπεράσματα λαμβάνουμε και για τα άλλα βίντεο.



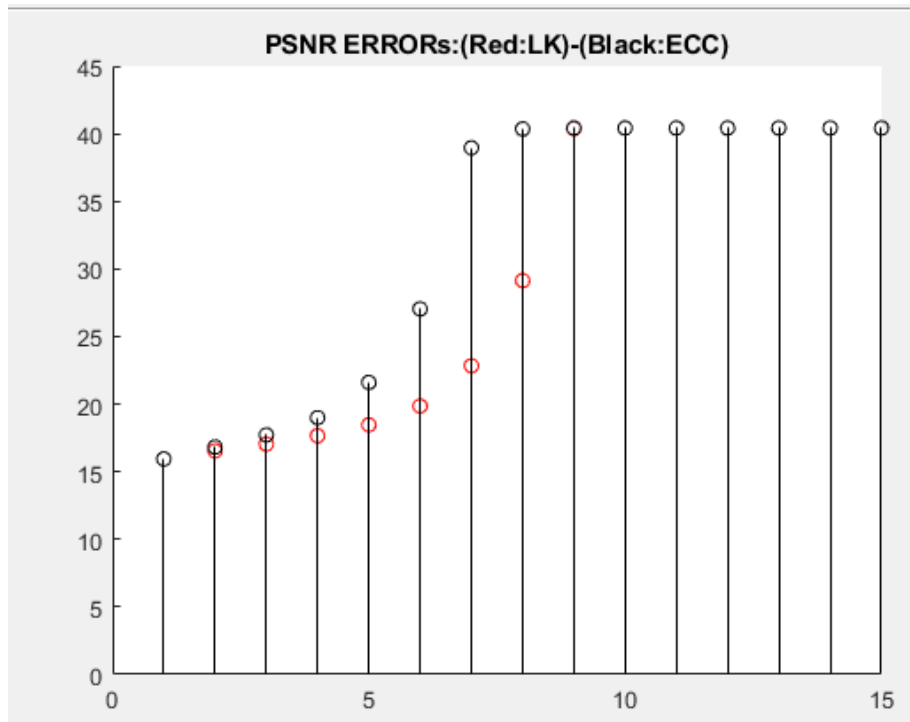
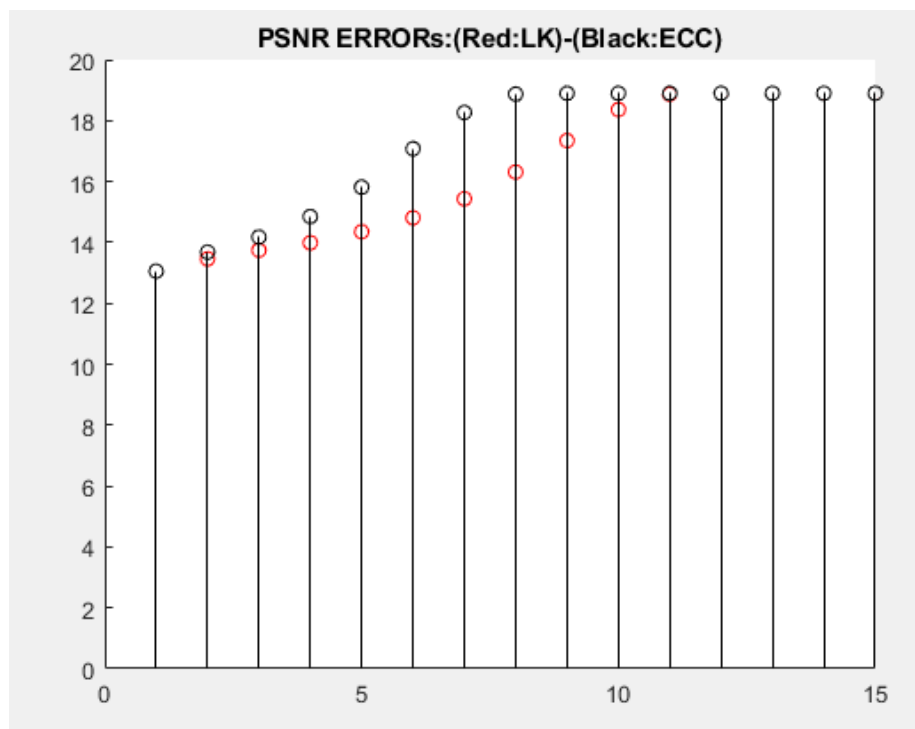
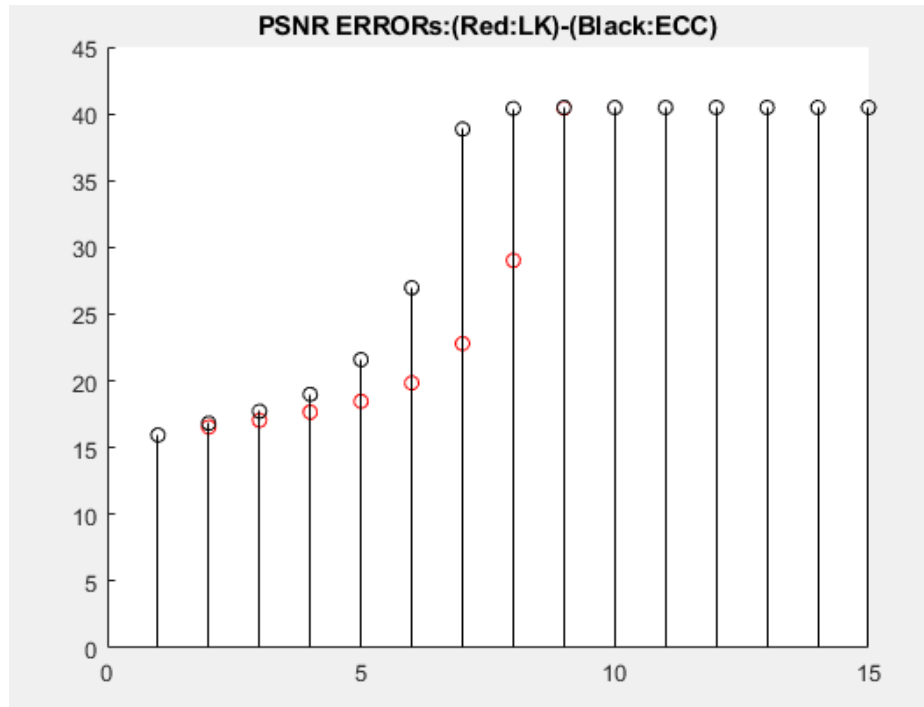


Image:

Τα επόμενα παραδείγματα μας είναι αύξηση contrast και brightness στην πρώτη περίπτωση και μείωση στην δεύτερη για το image του βίντεο 1 high.

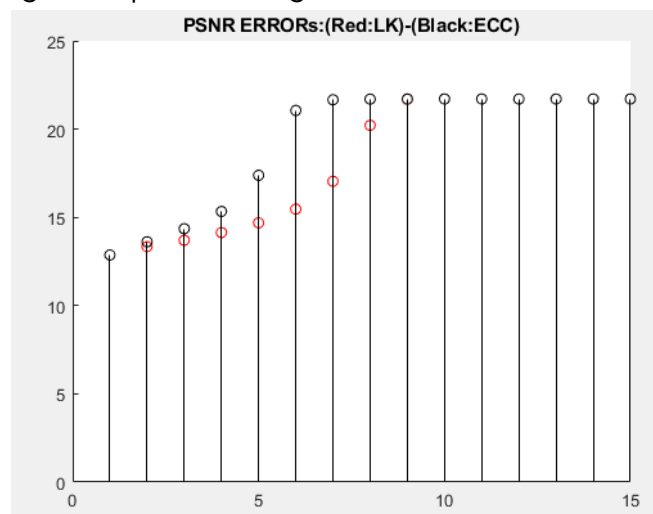


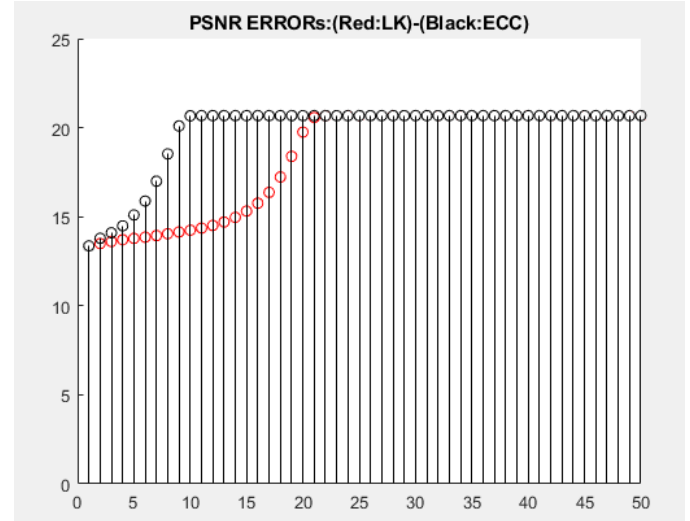
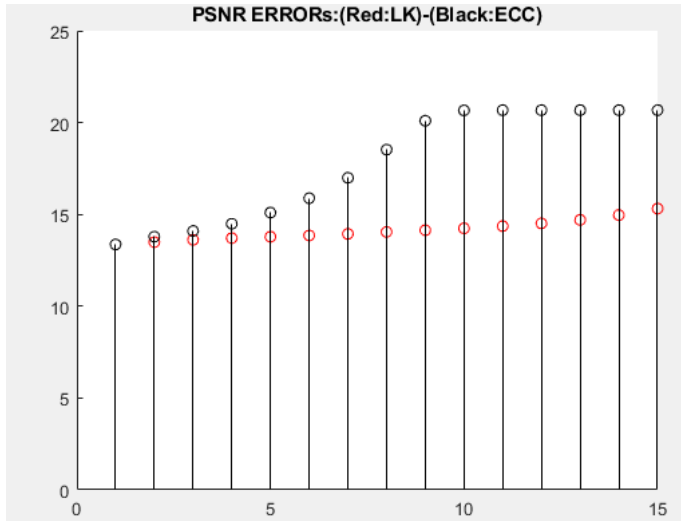
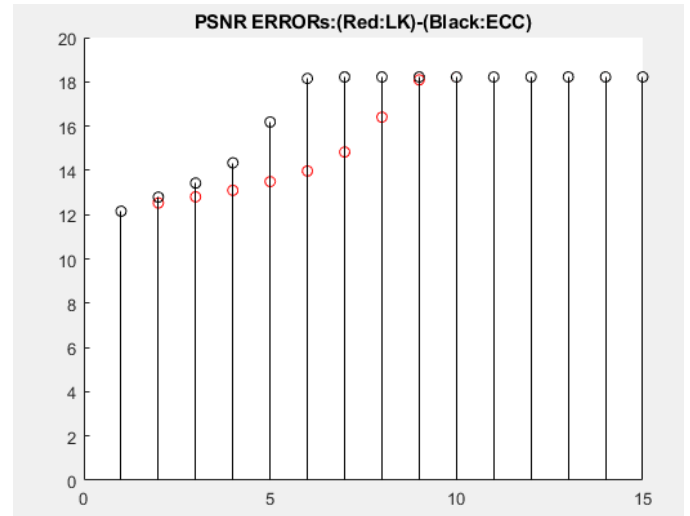
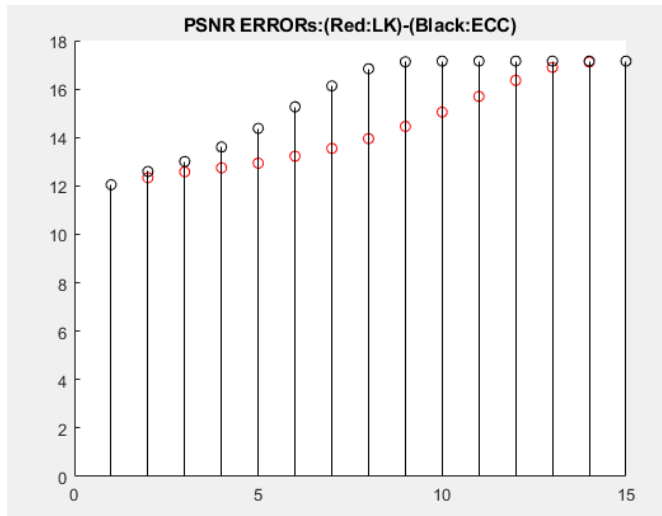


Σε αυτή την περίπτωση το image φαίνεται ότι έχει την δυνατότητα να αλλάξει ελάχιστα την ταχύτητα σύγκλισης του ecc κατά 1 η 2 επαναλήψεις, ενώ ο lk έως και 10 επαναλήψεις. Και πάλι παρατηρούμε ότι ο ecc επηρεάζεται ελάχιστα έως καθόλου ακόμα και αν αυξομειώσουμε κατά πολύ τις τιμές ενώ ο lk αλλάζει εντελώς.

Image και template:

Τα επόμενα παραδείγματα μας είναι αυξομειώσεις contrast και brightness για το template και image του βίντεο 1 high.

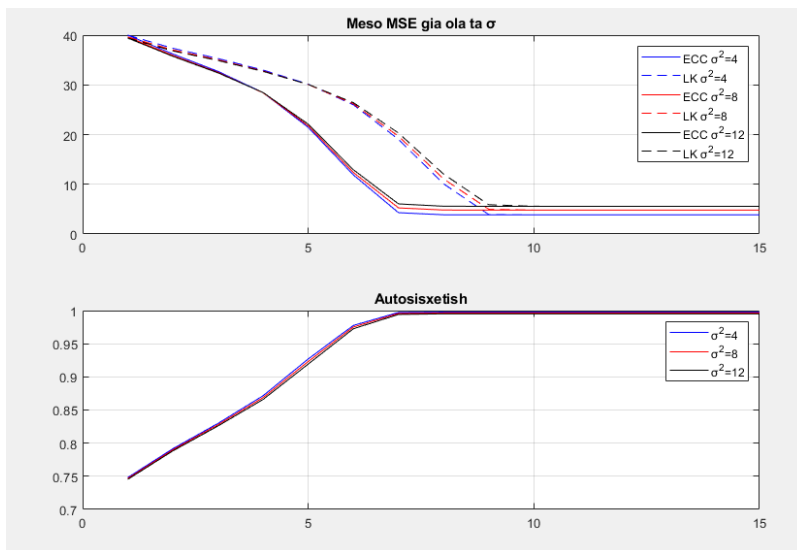




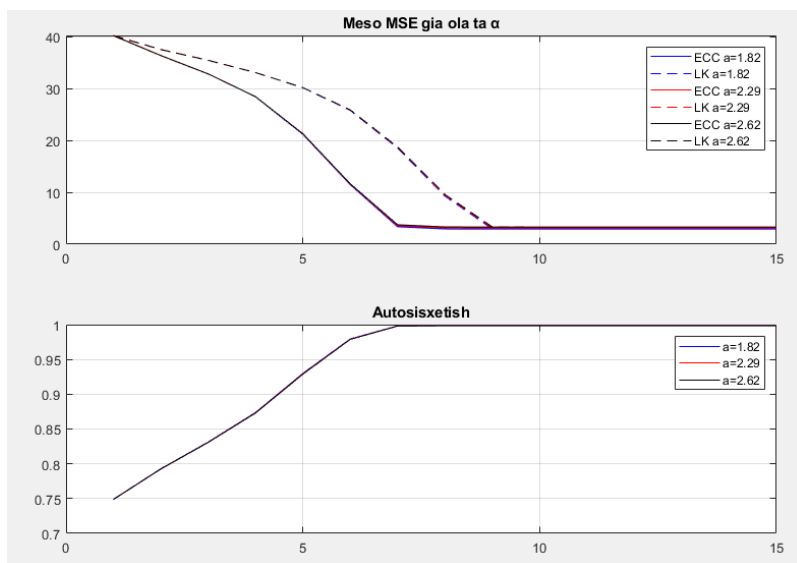
Παρατηρούμε ότι ακόμα και σε ακραίες αυξομειώσεις image και template ο ecc αλλάζει ελάχιστα ταχύτητα σύγκλισης κατά μερικές επαναλήψεις (2-3) ενώ ο lk σε μερικές περιπτώσεις αλλάζει πάνω από 15 επαναλήψεις.

6)

Gauss:



Uniform:



Παρατηρούμε ότι πάρα την παρουσία θορύβου, οι αλγόριθμοι καταφέρνουν να συγκλίνουν. Αυτό επιβεβαιώνεται από το μέσο τετραγωνικό σφάλμα που μειώνεται με κάθε επανάληψη και από την συσχέτιση του template και του image που βελτιώνεται με κάθε επανάληψη. Τέλος, ο ecc συγκλίνει πιο γρήγορα και έχει καλύτερη απόδοση στις ίδιες επαναλήψεις σε σχέση με τον lk.

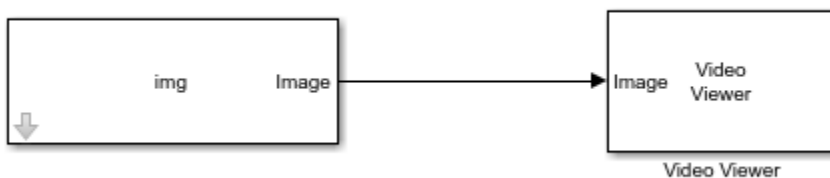
Μέρος Β

2)

Τρέχουμε πρώτα αυτές τις εντολές στο matlab (όχι στο Simulink)

```
load('img.mat');  
% First check image values  
min(img(:))  
max(img(:))  
  
% Scale image to proper range (0-255) and convert to uint8  
img = (img - min(img(:))) / (max(img(:)) - min(img(:))) * 255;  
img = uint8(img);  
  
% Check if it worked  
figure;  
imshow(img);  
  
ans =  
  
0  
  
ans =  
  
255
```

Υλοποιούμε αυτό το σύστημα για την προβολή εικόνας.



Το αποτέλεσμα είναι αυτό:



Για το βίντεο τρέχουμε πρώτα στο matlab αυτές τις εντολές.

```
load('img.mat')
if ~isa(img, 'uint8')
    img = uint8(img);
end

% Create a longer sequence - let's say 100 frames instead of 30
videoSequence = repmat(img, [1 1 1 100]); % Making 100 frames
```

Υλοποιούμε το παρακάτω σύστημα για την προβολή του βίντεο



3)

α)

Ο affine μετασχηματισμός στο R^2 αποτελείται από δύο συνιστώσες, έναν γραμμικό μετασχηματισμό Ax και μια μετάθεση t . Ο γραμμικός μετασχηματισμός, που αντιπροσωπεύεται από το μητρώο A , μπορεί να χειριστεί τη γεωμετρία της περιοχής S με διάφορους τρόπους, όπως

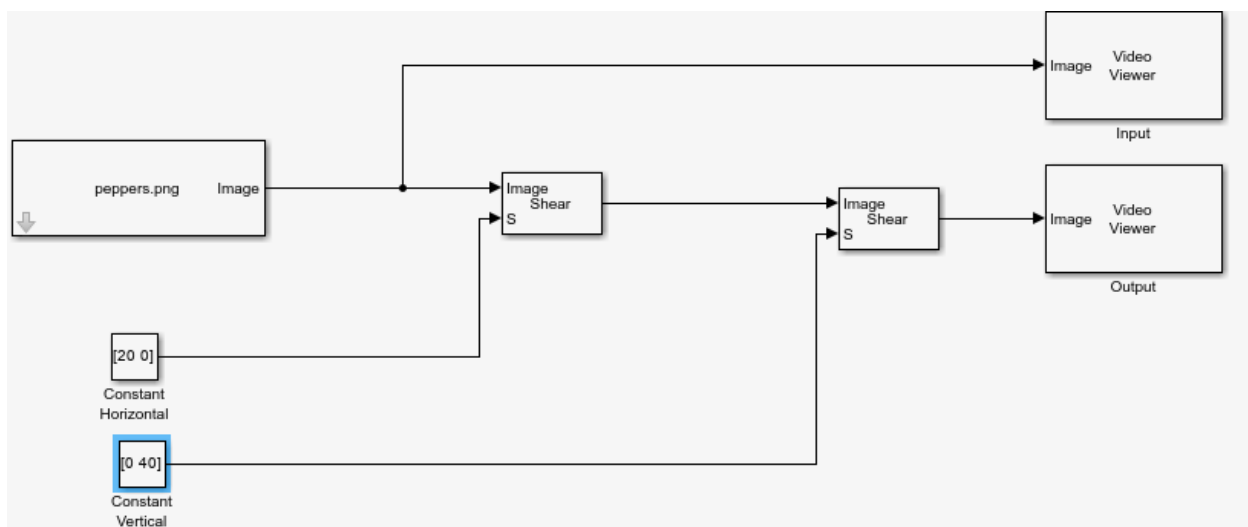
κλιμάκωση και περιστροφή. Το διάνυσμα μετάθεσης t μετατοπίζει την περιοχή σε διαφορετική θέση στο επίπεδο, αλλά δεν μεταβάλλει το μέγεθος ή το σχήμα της. Επομένως, η αλλαγή στην περιοχή S εξαρτάται αποκλειστικά από τις ιδιότητες του πίνακα γραμμικού μετασχηματισμού A .

Για μια περιοχή που οριοθετείται από μια κλειστή καμπύλη, το εμβαδόν της μπορεί να υπολογιστεί εξετάζοντας πώς ο μετασχηματισμός επηρεάζει τα απειροελάχιστα παραλληλόγραμμα που συνθέτουν την περιοχή. Όταν ο γραμμικός μετασχηματισμός A εφαρμόζεται σε οποιοδήποτε τέτοιο παραλληλόγραμμο, το εμβαδόν του κλιμακώνεται κατά έναν παράγοντα $|\det(A)|$.

Όταν $\det(A) < 0$, ο μετασχηματισμός περιλαμβάνει ανάκλαση, η οποία αντιστρέφει τον προσανατολισμό της περιοχής. Ωστόσο, επειδή μας ενδιαφέρει μόνο το μέγεθος της περιοχής και όχι ο προσανατολισμός, χρησιμοποιούμε την απόλυτη τιμή $|\det(A)|$.

b)

Αρχικά, τροποποιούμε το σύστημα όπως μας ζητείτε:



Το αποτέλεσμα είναι αναμενόμενο, η εικόνα μεταβάλλεται στις συντεταγμένες x και y του άξονα διατηρώντας όμως την παραλληλία των γραμμών που υπήρχε και ότι είναι ευθείες.

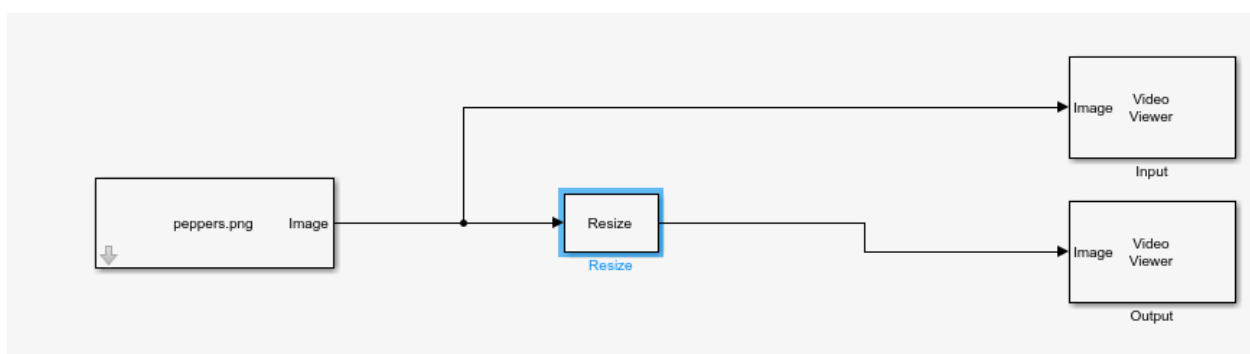
Αρχική εικόνα:



Shear:



Για το resize υλοποιούμε το εξής σύστημα:

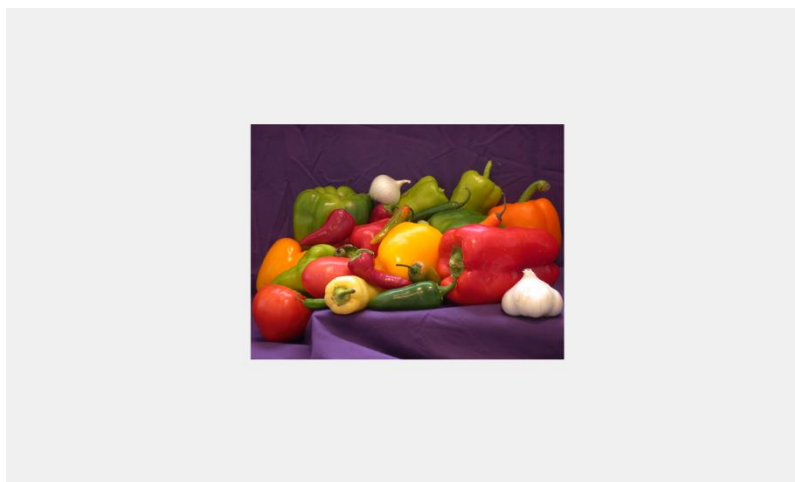


με τις εξής παραμέτρους

Main	Data Types
Parameters	
Specify:	Output size as a percentage of input size
Resize factor in percentage:	[200 200]
Interpolation method:	Bilinear

Ουσιαστικά διπλασιάσαμε το μέγεθος της εικόνας

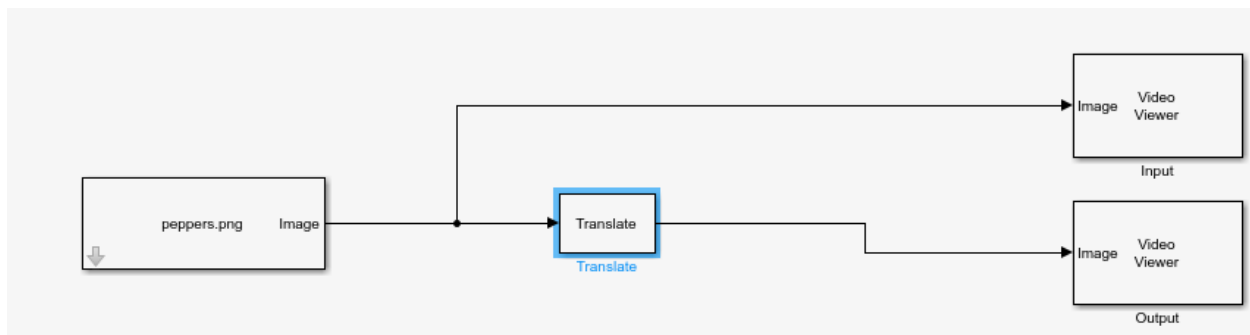
Αρχική:



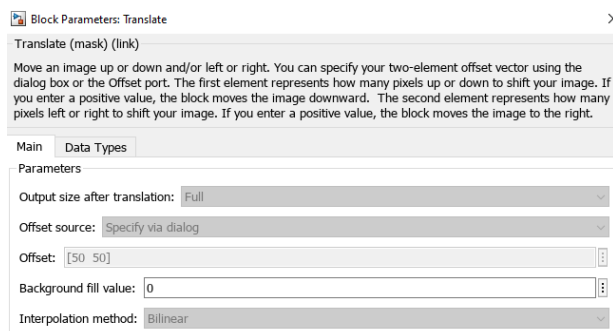
Resize:



Για το translate υλοποιούμε το εξής σύστημα:



Ουσιαστικά, μετακινεί την εικόνα χωρίς να αλλάξει το σχήμα ή το μέγεθος προς την κατεύθυνση που εμείς ορίζουμε. Θέτουμε η εικόνα να μετακινηθεί κατά 50 πιξελ και στους δυο άξονες.



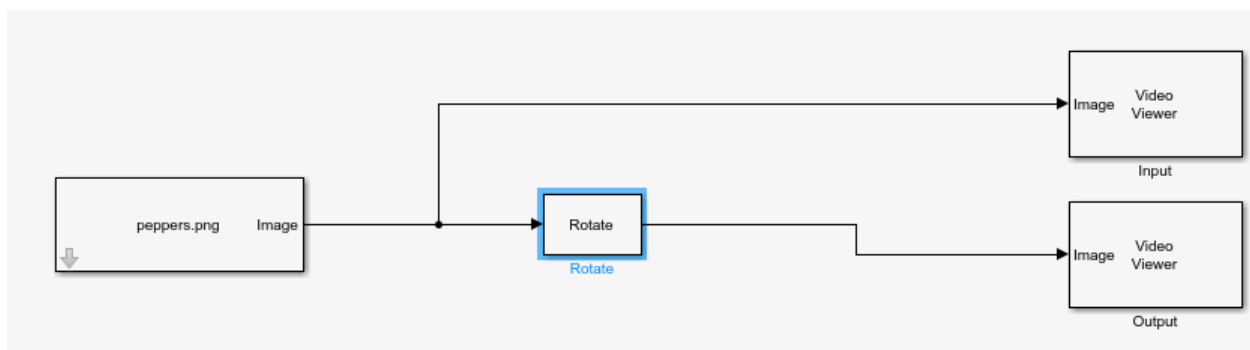
Αρχική:



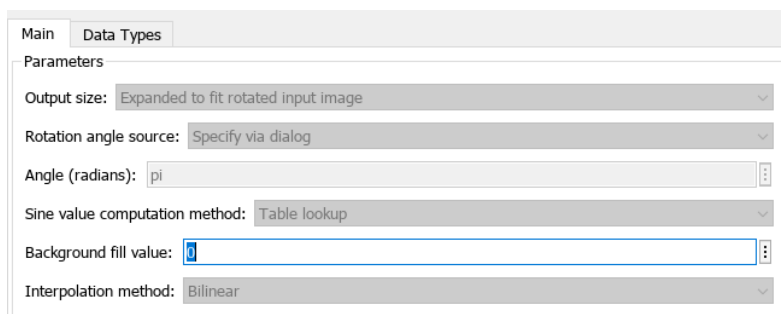
Translate:



Για το rotate υλοποιούμε το εξής σύστημα:



Περσστρέφει την εικόνα μας ανάλογα με το πόσες μοίρες θέλουμε. Στην δικιά μας περίπτωση, θέτουμε 180 μοίρες περιστροφή.



Αρχική:

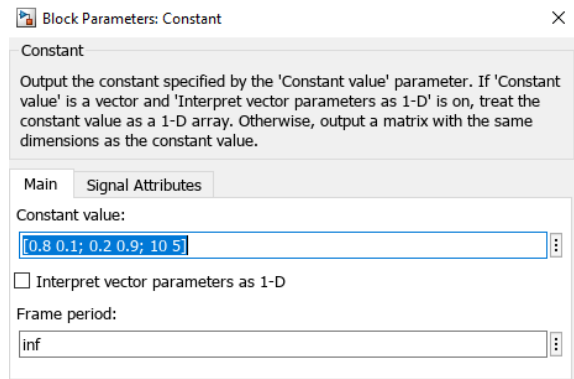


Rotate:



c)

Ο μετασχηματισμός που ζήτησε είναι ουσιαστικά η συλλογή όλων των παραπάνω μετασχηματισμών, εκτός από τον rotate. Συγκεκριμένα, τα διαγώνια στοιχεία είναι scaling, τα αντιδιαγώνια είναι shear και η τελευταία στήλη είναι transform. Άρα, έχουμε μια εικόνα η οποία θα είναι μικρότερη και στους 2 άξονες, έχει υποστεί αλλαγές στις τιμές των 2 αξόνων από το shear και τέλος έχει μετατοπιστεί λόγω του translate.



Σαν input βάλαμε το βίντεο που δημιουργήσαμε στην αρχή.

Αρχικά:



Μετασχηματισμός:



d)

Τα μητρώα P1 και P2 που δίνονται εκτελούν τους ίδιους μετασχηματισμούς με τους affine που περιγράψαμε προηγούμενος, με την προσθήκη ότι η τελευταία γραμμή περιέχει μετασχηματισμούς που αλλάζουν την γωνιά βλέψης μας και έχει επίσης το ολικό scaling που είναι 1.

$$H = \begin{bmatrix} h_1 & h_4 & h_7 \\ h_2 & h_5 & h_8 \\ h_3 & h_6 & h_9 \end{bmatrix}$$

where $h_1, h_2, \dots, h_9$ are transformation coefficients.

The value of the pixel located at $[x, y]$ in the input image determines the value of the pixel located at $[\hat{x}, \hat{y}]$ in the output transformed image.

The relationship between the input and output point locations is defined by:

$$\begin{cases} \hat{x} = \frac{xh_1 + yh_2 + h_3}{xh_7 + yh_8 + h_9} \\ \hat{y} = \frac{xh_4 + yh_5 + h_6}{xh_7 + yh_8 + h_9} \end{cases}$$

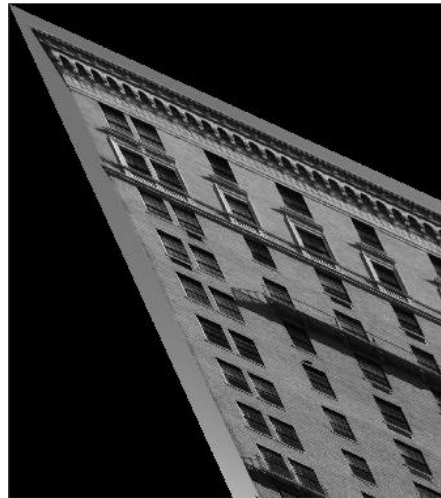
Οι δεδομένες τιμές δεν δουλεύουν στο block warp στις νεότερες εκδόσεις matlab (αντικαταστάτης του apply geometric transformation), άρα κάνουμε παραδοχή και αλλάζουμε τις τιμές με διαφορετικές. Εισάγουμε τις εξής τιμές στο block constant:

P1:

Main Signal Attributes

Constant value:

[1.1 0.5 0; 0.5 1.1 0; 0.1 0.05 1]



P2:

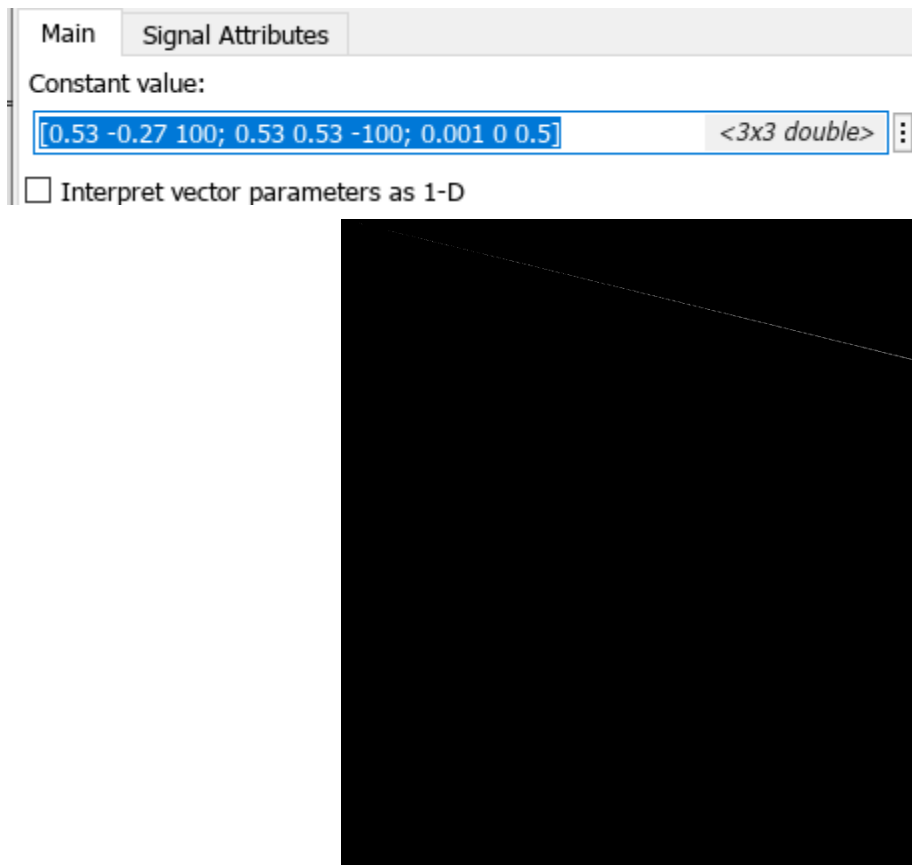
Main Signal Attributes

Constant value:

[1.1 0.3 0; 0.3 0.9 0; 0.15 0.1 1]



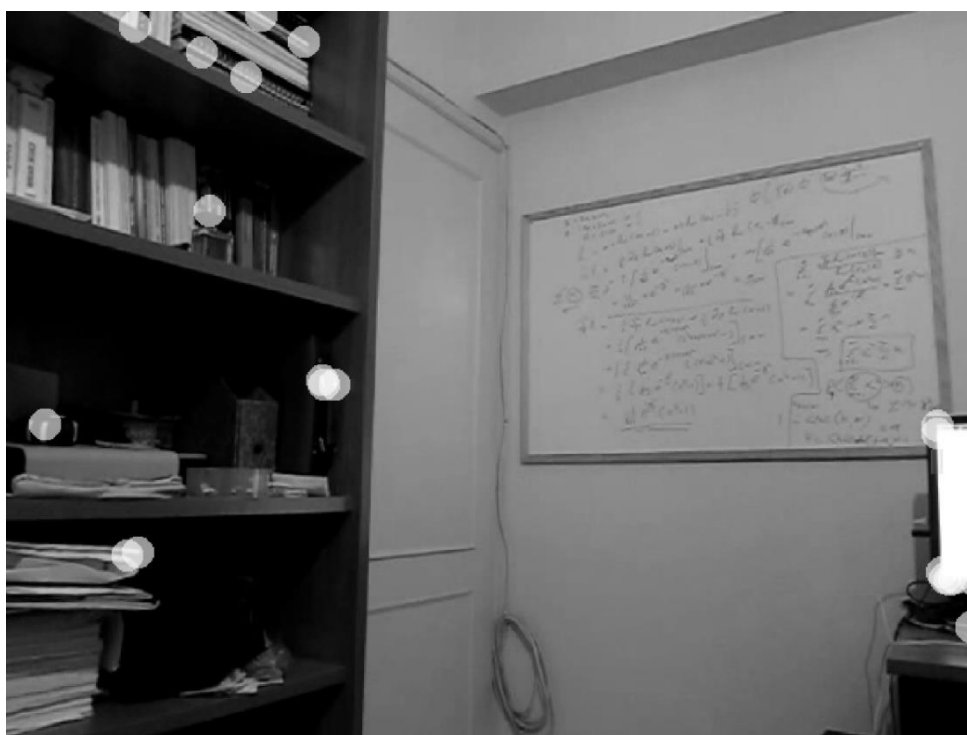
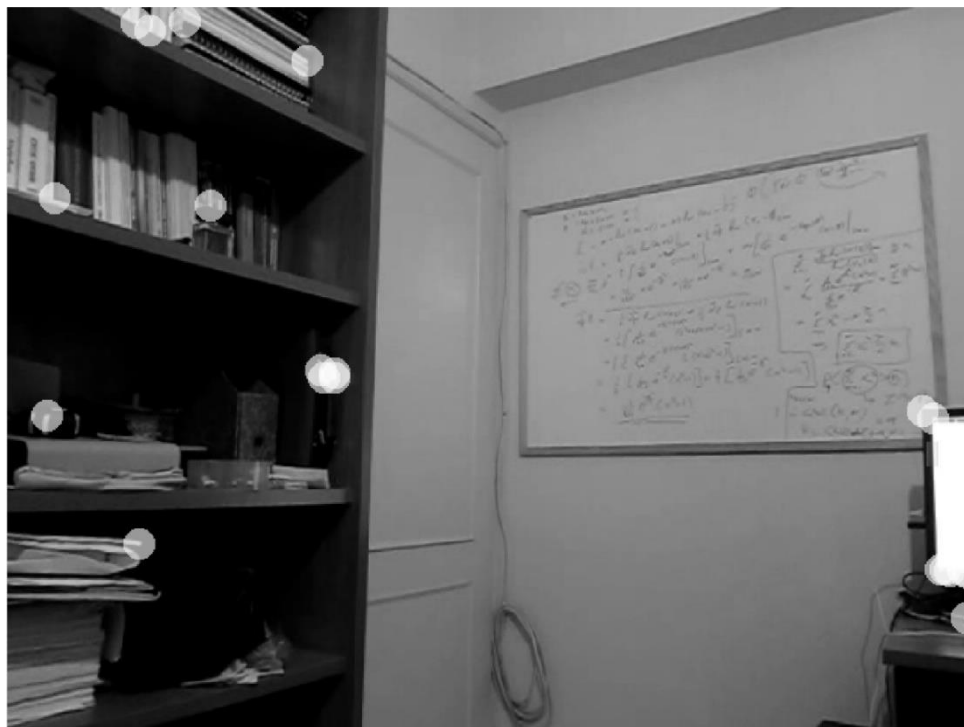
X,Y:

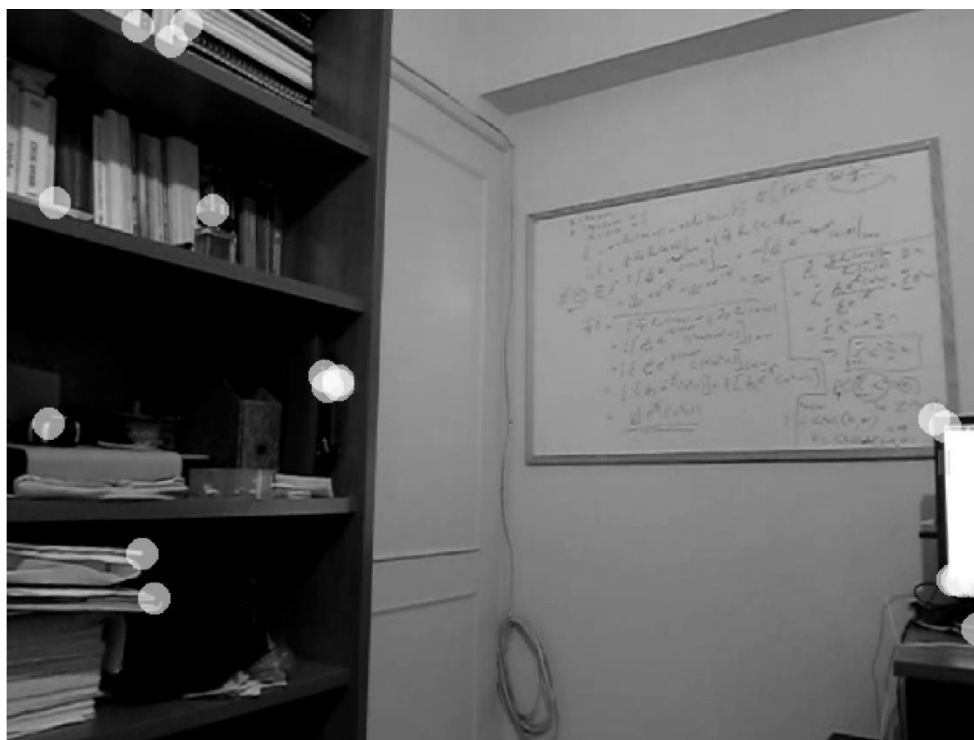


Από τις τιμές των μητρώων παρατηρούμε ότι το P1 εισάγει μεγάλες τιμές shear ειδικά στον άξονα y και επίσης μεγάλο resize στον άξονα y . Το ακριβώς αντίθετο συμβαίνει για το P2. Για τα x,y που δίνονται βλέπουμε ότι οι μεταμορφώσεις που εισάγουν είναι θεωρητικά λιγότερες επειδή έχει μικρότερες τιμές συντελεστών από το P1 και P2 αλλά πρακτικά το βλέπουμε από μια περίεργη γωνία.

4)

Η πρώτη εικόνα είναι harris, η δεύτερη είναι drummond και η τρίτη είναι shi.

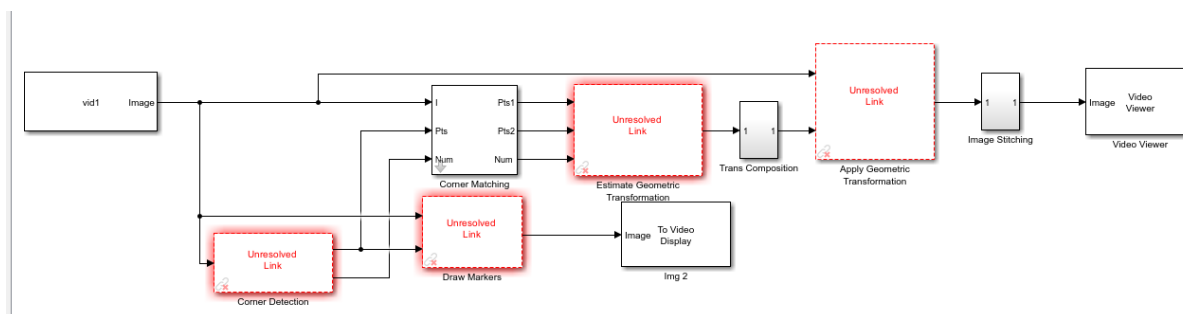




Από εικόνες παρατηρούμε τα αναμενόμενα αποτελέσματα. Ο harris εντοπίζει με ακρίβεια όλες τις γωνίες, καθώς έχει τους περισσότερους υπολογισμούς. Ο `drummond` εντοπίζει μερικές γωνίες λανθασμένα, το οποίο δικαιολογείται καθώς είναι φτιαγμένος για υψηλότερη ταχύτητα. Τέλος, ο `shi` πλησιάζει σε απόδοση τον `harris` χρησιμοποιώντας απλούστερες πράξεις.

5)

Το δεδομένο μοντέλο δεν γίνεται να υλοποιηθεί στην τρέχον έκδοση matlab



Απαντάμε στα ερωτήματα με βάση την θεωρία.

Ο RANSAC είναι ένας εύρωστος αλγόριθμος προσαρμογής μοντέλου που έχει σχεδιαστεί για την εκτίμηση των παραμέτρων ενός μοντέλου από ένα σύνολο δεδομένων που περιέχει σημαντικό ποσοστό ακραίων τιμών. Λειτουργεί επιλέγοντας επαναληπτικά τυχαία υποσύνολα σημείων δεδομένων, προσαρμόζοντας ένα μοντέλο σε αυτά τα σημεία και στη συνέχεια αξιολογώντας πόσο καλά τα υπόλοιπα δεδομένα ταιριάζουν στο μοντέλο. Τα σημεία που προσαρμόζονται στο μοντέλο εντός μιας προκαθορισμένης ανοχής ταξινομούνται ως μηδενικά, ενώ τα υπόλοιπα αντιμετωπίζονται ως ακραία. Ο αλγόριθμος συνεχίζεται για έναν καθορισμένο αριθμό επαναλήψεων ή μέχρι να βρεθούν επαρκή αριθμό μη ακραίων σημείων.

Ο αλγόριθμος Least Median of Squares είναι μια άλλη ισχυρή προσέγγιση για την προσαρμογή μοντέλων, αλλά ακολουθεί διαφορετική διαδρομή από τον RANSAC. Ελαχιστοποιεί τη διάμεσο των τετραγωνικών καταλοίπων μεταξύ των σημείων δεδομένων και του μοντέλου, άρα το αποτέλεσμα δεν επηρεάζεται υπερβολικά από ακραίες τιμές. Επιλέγει επίσης τυχαία υποσύνολα σημείων δεδομένων για τον υπολογισμό μοντέλων, αλλά αξιολογεί αυτά τα μοντέλα με βάση το διάμεσο τετραγωνικό σφάλμα, επιλέγοντας το μοντέλο με το μικρότερο διάμεσο υπόλοιπο.

Οι μετασχηματισμοί συγγένειας διατηρούν ορισμένες ιδιότητες σχημάτων και δομών σε μια εικόνα. Συγκεκριμένα, οι συγγενείς μετασχηματισμοί διατηρούν την παραλληλία μεταξύ των γραμμών και τις σχετικές αναλογίες των σημείων κατά μήκος μιας γραμμής.

Αντίθετα, οι προβολικοί μετασχηματισμοί διατηρούν την ευθύτητα, δηλαδή οι ευθείες γραμμές στην αρχική εικόνα παραμένουν ευθείες μετά τον μετασχηματισμό, αλλά δεν διατηρούν την παραλληλία ή τις αναλογίες.